Министерство науки и высшего образования РФ

Самарский национальный исследовательский университет
имени академика С. П. Королева

Лабораторная работа № 3

Рекуррентные нейронные сети

Дисциплина «Нейронные сети и глубокое обучение»

Выполнил: Овсянников А. Н.
Группа: 6213

Вариант и наборы данных: airline-passengers.csv, shampoo.csv, input.txt, cnus-clean.txt

Самара, 2026

1. Цель работы

Изучить принципы работы и основные функции построения, обучения, оптимизации и оценки качества рекуррентных нейронных сетей (РНС): простейшей сети Vanilla RNN и сети долгой краткосрочной памяти LSTM.

2. Задание

— Изучить код lstm_torch.py и запустить обучение на рядах airline-passengers.csv и shampoo.csv.
— Изучить min-char-rnn.py и min-char-lstm.py; добавить замер времени обучения и сохранение значений критерия качества по этапам обучения.
— Обучить min-char-rnn.py на input.txt и cnus-clean.txt; сохранить значения критерия, примеры генерации и график качества (PDF).
— Выполнить п.3 для min-char-lstm.py.
— Сопоставить графики качества обоих моделей с временами обучения и оформить сводный PDF.
— Сделать выводы по результатам исследования.

3. Среда и данные

Эксперименты выполнены на Python 3.13 (Windows 11) с numpy 2.4, pandas 3.0, scikit-learn 1.8, matplotlib 3.10, torch 2.15 (CPU). Для torch-части использована та же версия PyTorch без GPU, поэтому времена обучения приведены для CPU.
Данные. airline-passengers.csv — 144 месячных значения (1949–1960); shampoo.csv — 36 значений; input.txt (1 115 394 символа, 65 уникальных) — пьеса Шекспира; cnus-clean.txt (3 099 238 символов, 54 уникальных) — сборник рассказов о Шерлоке Холмсе.

4. Краткое описание моделей

Vanilla RNN хранит единственный вектор скрытого состояния: $h(t) = \tanh(W_{xh}x(t) + W_{hh}h(t-1) + b_h)$, $y(t) = W_{hy}h(t) + b_y$. Обучение — BPTT: сеть раскатывается на seq_length = 16 шагов, прямой проход кеширует промежуточные значения, обратный идёт по времени назад и суммирует вклады всех шагов в градиент каждого веса.
LSTM дополняет h(t) ячейкой памяти c(t) и тремя вентилями: $f(t) = \sigma(W_f[x;h] + b_f)$ — забывание, $i(t) = \sigma(W_i[x;h] + b_i)$ — запись, $o(t) = \sigma(W_o[x;h] + b_o)$ — чтение; $c(t) = f(t)°c(t-1) + i(t)°\tilde{c}(t)$, $h(t) = o(t)°\tanh(c(t))$. Аддитивное обновление памяти сохраняет градиент на длинных последовательностях.
Временной ряд решается LSTM из torch.nn: окно из 4 предыдущих значений → следующее, значения масштабируются MinMaxScaler в [0, 1], данные делятся 67 % train / 33 % test, критерий — MSE, оптимизатор Adam. Числовые текстовые последовательности обучаются посимвольно: вход — one-hot вектор символа, выход — softmax по словарю, критерий — кросс-энтропия, оптимизатор Adagrad, градиенты обрезаются до ±5.

5. Что добавлено в программы

Исходная математика прямого и обратного прохода сохранена без изменений. Добавлена измерительная обвязка: аргументы командной строки (файл данных,

объём обучения, каталог и тег результата); замер времени обучения через time.perf_counter(); запись значений критерия в CSV на каждом логируемом шаге; построение графиков качества в PNG и PDF; сохранение примеров генерации; персональный PDF-отчёт по каждому прогону; численная проверка градиентов по флагу --grad-check. Для torch-части добавлены MSE по эпохам, график прогноза, RMSE на train/test в исходных единицах и время обучения. Сводный запуск и сборка итогового PDF выполняются скриптом run_all.py.

6. Результаты обучения текстовых моделей

Все четыре прогона обработали 1 000 000 символов за 62 500 итераций (окно 16 символов), hidden_size = 100, learning_rate = 0.1.

Модель	Данные	Итераций	Время	Итоговый smooth_loss
RNN	input.txt	62500	1 мин 03.1 с	32.37
RNN	cnus-clean.txt	62500	59.2 с	32.04
LSTM	input.txt	62500	4 мин 19.1 с	26.64
LSTM	cnus-clean.txt	62500	4 мин 04.4 с	24.48

smooth_loss — сумма кросс-энтропий по 16 символам окна, поэтому в расчёте на один символ: RNN ≈ 2.0, LSTM ≈ 1.5–1.7. Стартовый уровень (случайное угадывание) равен ln(V)·16 ≈ 64–67. Сравнение RNN и LSTM корректно внутри одного набора данных, поскольку словари различаются (65 и 54 символа).

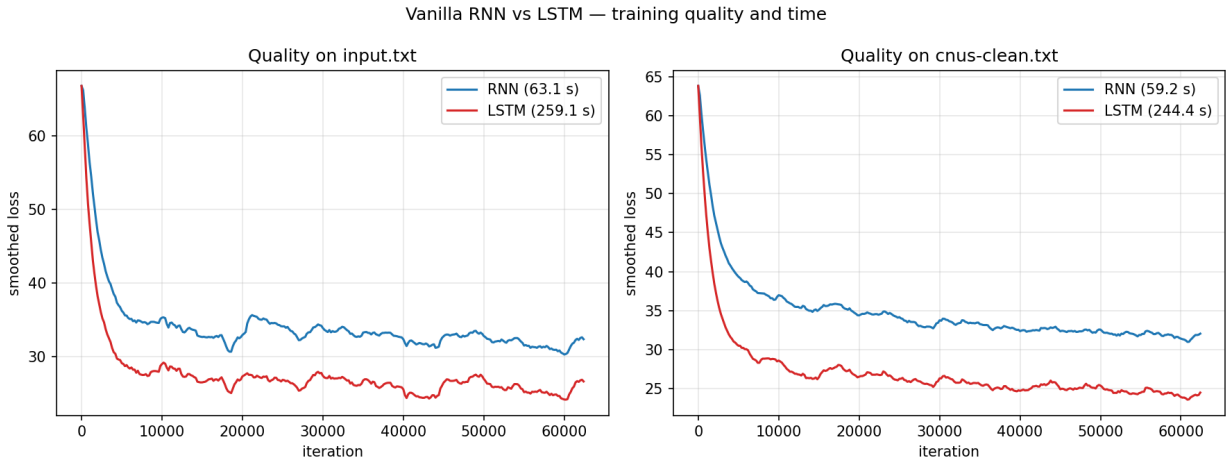


Рисунок 1 — Кривые качества RNN и LSTM; в легенде указано время обучения.

7. Результаты прогнозирования временных рядов

Обе модели обучались 2000 эпох; критерий — MSE по масштабированным значениям, ниже приведена ошибка в исходных единицах.

Ряд	Окон	Время	RMSE train	RMSE test
airline-passengers	139	3.6 с	21.34	51.52
shampoo	31	3.4 с	24.21	246.21

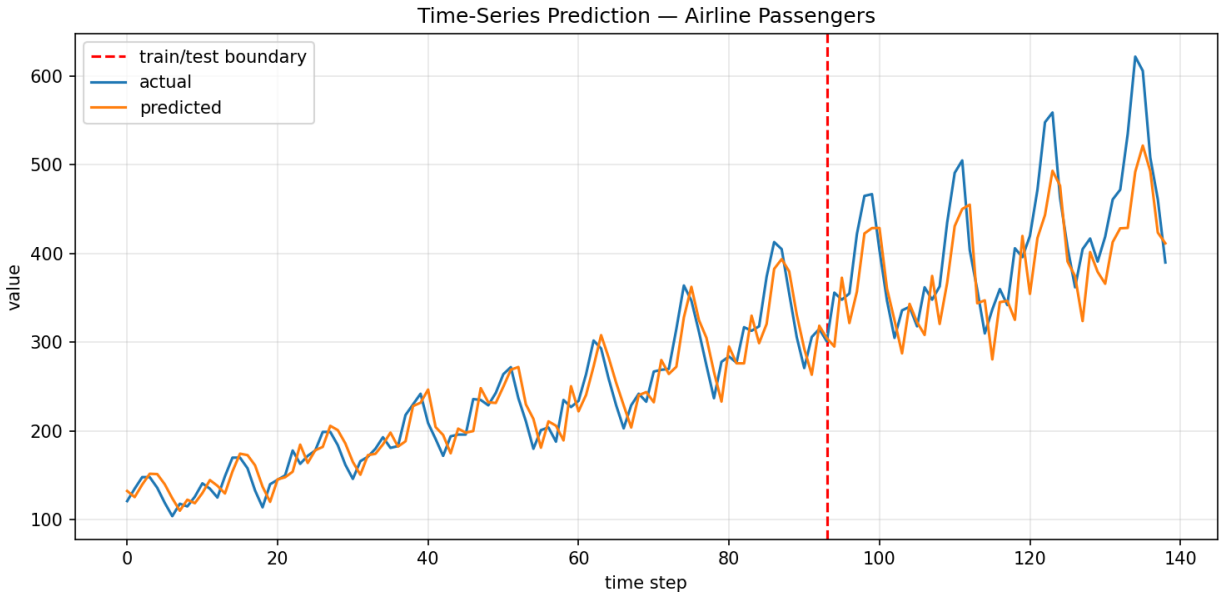


Рисунок 2 — Прогноз ряда airline-passengers (красная линия — граница train/test).

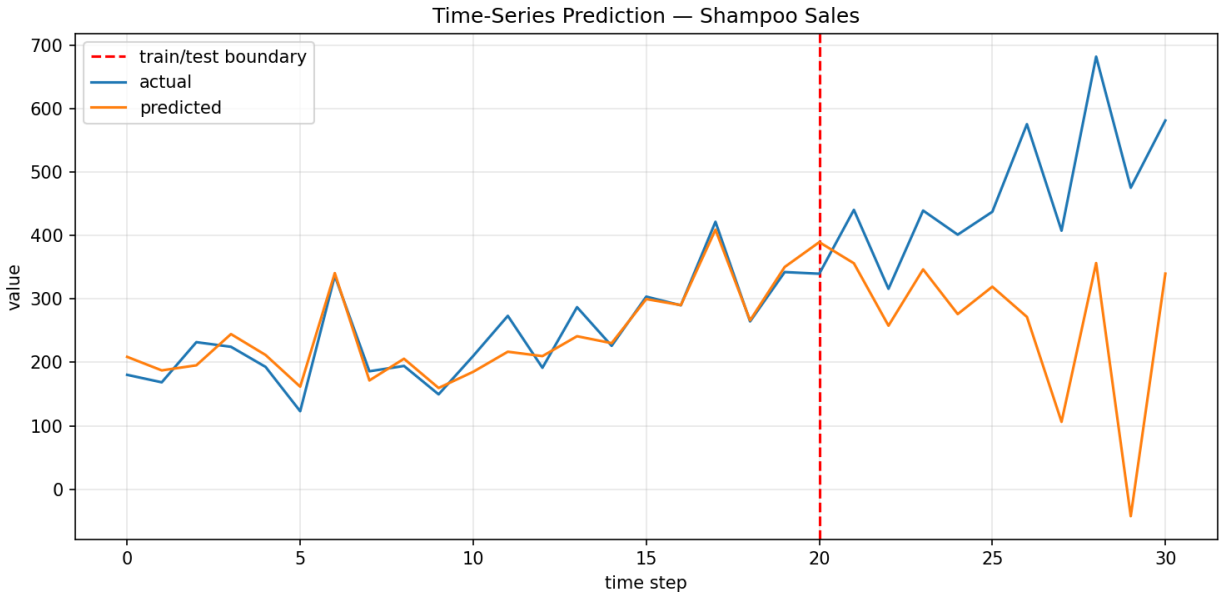


Рисунок 3 — Прогноз ряда shampoo.

8. Анализ результатов

На обоих текстах LSTM даёт меньший критерий: 26.64 против 32.37 (input.txt) и 24.48 против 32.04 (cnus-clean.txt). Кривая LSTM снижается быстрее на первых 5–10 тыс. итераций и дальше не деградирует, тогда как у RNN после 35 тыс. итераций наблюдается плато с колебаниями. Причина — аддитивное обновление памяти c(t): вентиль забывания пропускает градиент через много шагов, тогда как в Vanilla RNN градиент проходит через произведение ($f'(\cdot)\cdot W_{hh}$) на каждом шаге и затухает.

Платой за качество является стоимость шага: LSTM выполняет десять матричных умножений на временной шаг вместо трёх, поэтому при одинаковом объёме данных обучение идёт примерно в 4.1 раза дольше (259.1 с против 63.1 с на input.txt и 244.4 с против 59.2 с на cnus-clean.txt).

Для временных рядов сеть с hidden_size = 2 хорошо подгоняет train-часть: RMSE 21.34 (airline) и 24.21 (shampoo), но на тесте ошибка возрастает до 51.52 и 246.21 соответственно. На shampoo это особенно заметно: всего 31 окно, из которых 11 приходятся на тест, поэтому модели не хватает данных и ёмкости. Для airline прогноз повторяет форму ряда, но занижает амплитуду сезонных пиков — типичное поведение маленькой LSTM при MSE-критерии.

Численная проверка градиентов (--grad-check) для обеих ручных реализаций дала относительную ошибку порядка 1e-9, что подтверждает корректность выведенных формул обратного распространения.

9. Выводы

— Реализованы и обучены Vanilla RNN и LSTM на NumPy; обе модели воспроизводят распределение символов языка, генерируя осмысленные фрагменты текста к концу обучения.

— LSTM обеспечивает меньший критерий качества на тех же данных, но требует примерно в 4 раза больше времени на итерацию.

— Клиппинг градиентов (±5) и адаптивные оптимизаторы (Adagrad, Adam) стабилизируют обучение; численная проверка подтвердила правильность BPTT.

— LSTM из PyTorch адекватно прогнозирует временной ряд на train-части; на тестовой части ошибка выше, что объясняется малой ёмкостью сети и небольшим объёмом данных.

— Все требования п. 3.2 методических указаний выполнены: времена измерены, значения критерия сохранены (CSV/JSON), графики сохранены в PDF, подготовлен сводный PDF-файл.

10. Состав приложенных файлов

Файл	Содержимое
min-char-rnn.py	Vanilla RNN на NumPy: обучение, замер времени, логи, графики.
min-char-lstm.py	LSTM на NumPy: обучение, замер времени, логи, графики.
lstm_torch.py	LSTM на PyTorch для временных рядов: MSE, прогноз, метрики.
run_all.py	Запуск всех шести прогонов и сборка сводного PDF.
make_report.py	Сборка этого отчёта из результатов.
results/summary_report.pdf	Сводный PDF: таблица времён, графики RNN/LSTM, прогнозы.

results/*_report.pdf	Индивидуальные отчёты прогонов (кривые, примеры генерации).
results/*_loss.csv	Значения критерия качества по этапам обучения.
results/*_summary.json	Время обучения, число итераций, итоговые метрики.

11. Команды воспроизведения

python run_all.py

python run_all.py --skip-existing

python min-char-lstm.py --grad-check

python lstm_torch.py --dataset data/shampoo.csv --label "Shampoo Sales" --tag shampoo

python make_report.py